

TriviaQA: Open-domain Question Answering Report

Tommaso Giorgini

EHU/UPV - University of the Basque Country

Abstract

This work investigates retrieval-augmented generation (RAG) for open-domain question answering on the TriviaQA dataset. We develop a complete RAG pipeline consisting of document preprocessing, passage chunking, sparse and dense retrieval methods, reranking, and answer generation with instruction-tuned large language models. The system retrieves relevant evidence from Wikipedia and conditions a generator model on both the question and the retrieved passages to produce grounded answers. We compare multiple retrieval strategies as well as several compact LLMs, and we analyze how factors such as different retrieval methods, context window size, and prompt design influence downstream answer quality. This report outlines the methodology, experimental setup, and evaluation framework used to study these components.

Link of the project's notebook hosted on Google Colab:

Project:

https://colab.research.google.com/drive/1_stgRkaDRQYkhEhGg6pLPGe48qnkTHef?usp=sharing

Data Preprocessing:

<https://colab.research.google.com/drive/12sLHN2lH-Z8kdu29Zms1zZCN2CnZl80t?usp=sharing>

1 Introduction

Open-domain question answering (ODQA) (Chen et al., 2017) aims to answer factoid questions using large, unstructured knowledge sources such as Wikipedia or the web. Recent large language models (LLMs) can often answer such questions directly from their parametric knowledge, but they also suffer from well-known limitations: they may hallucinate facts, their knowledge quickly becomes

outdated, and updating them via full fine-tuning is computationally expensive.

Retrieval-Augmented Generation (RAG) addresses these issues by combining an information retriever with a generative model. Given a question, the retriever first searches an external corpus for relevant passages. The generator then infers on both the question and the retrieved passages to produce an answer that is grounded in evidence, reducing hallucinations and avoiding costly task-specific fine-tuning. This makes RAG particularly attractive for knowledge-intensive tasks such as ODQA.

In this project, we build and evaluate a RAG-based QA system on the TriviaQA dataset (Joshi et al., 2017), a large-scale collection of trivia questions with associated evidence documents from Wikipedia and web search. We focus on the Wikipedia setting, where documents are often long and must be split into passages that can be efficiently indexed and retrieved under the context-length constraints of current LLMs.

Our main goals are to

1. implement a complete RAG pipeline for TriviaQA with separate retrieval and generation components,
2. compare different retrieval strategies and generative models under realistic resource constraints,
3. analyze how retrieval quality and prompt design affect answer accuracy.

The remainder of this paper describes our methodology, presents our experimental results, and summarizes the main findings and limitations of our approach.

2 Methodology

2.1 Task and Setup

The main objective of this work is to create a question-answering system as retrieval-augmented generation (RAG) for answering questions from open domains on TriviaQA. Given a question, a retriever selects relevant passages from a large Wikipedia collection, and a generative large language model (LLM) produces a short answer conditioned on the question and the retrieved passages. The LLM is used in inference-only mode; we do not fine-tune its parameters, but improve task performance only through retrieval and prompting.

2.2 Dataset and Splits

We use the RC version of TriviaQA (Joshi et al., 2017), which contains over 650K question-answer-evidence triples and 95K human-written trivia questions. Each question is paired with a canonical answer, a set of aliases, and on average about six independently collected evidence documents. In this work we restrict ourselves to the Wikipedia portion of the dataset: evidence comes only from Wikipedia articles referenced in the EntityPages field and stored under evidence/wikipedia.

Following the project specification, we construct our splits as follows:

- **Validation:**

We take qa/wikipedia-train.json and use the first 7,900 question-answer pairs (with all their evidence documents) as a validation set. The remaining examples form the training set, not used in this work.

- **Test:**

We use qa/verified-wikipedia-dev.json as our test set with human-verified questions, answers, and Wikipedia evidence.

2.3 Preprocessing and Retrieval

The original evidence documents are full Wikipedia articles, which are too long to be directly fed into the generator. We therefore load the raw text for each article, remove markup and excessive whitespace, and split the article into overlapping passages (chunks) using a sliding window over tokenized text. After a brief empirical exploration based on Recall@K, we fixed the chunk length to 350 words and the overlap to 70 words, as this configuration

provided a good trade-off between retrieval effectiveness and index size. Each chunk is thus created with a stride smaller than its length, so that adjacent chunks overlap and increase the chance that complete answer spans appear inside at least one passage. Each passage is stored as an independent retrieval unit, together with metadata linking it to the original document.

We consider two families of retrieval models: a sparse BM25 retriever (Robertson and Zaragoza, 2009) and a dense neural retriever based on Sentence-BERT (Reimers and Gurevych, 2019), followed by a Cross-Encoder re-ranking step (Reimers and Gurevych, 2019).

First, we test a BM25 retriever (Robertson and Zaragoza, 2009). All Wikipedia-derived passages are segmented into overlapping chunks and indexed with a sparse BM25 retriever. At inference time, the question text is used as a query, and BM25 ranks passages by a weighted sum over shared terms that combines term frequency saturation, inverse document frequency, and document-length normalization. This yields an efficient lexical baseline that prioritizes passages containing the most informative query terms.

In addition, we experiment with a dense retrieval model, where both questions and passages are encoded into a shared vector space using a neural encoder, and retrieval is performed via nearest-neighbor search in the embedding space. For this, we use the sentence-transformers/all-MiniLM-L6-v2 checkpoint from the Sentence-Transformers library (Reimers and Gurevych, 2019). Dense retrieval relaxes the need for exact token overlap and can capture semantic similarity between questions and passages even when they do not share surface forms.

Lastly, to further improve the ranking quality of the retrieved passages, we apply a Cross-Encoder-based re-ranking step on top of the initial BM25 candidates. For each question, we take the top- n retrieved passages and feed each question-passage pair into a Cross-Encoder model, which directly predicts a relevance score. Concretely, we use the cross-encoder/ms-marco-MiniLM-L-6-v2 model (Reimers and Gurevych, 2019), trained on MS MARCO-style relevance signals (Nguyen et al., 2016). The passages are then re-sorted according to these scores, ensuring that the passages passed to the generator are as relevant and informative as

possible with respect to the question. A detailed evaluation of these retrieval methods is reported in Section 3.

2.4 Generator and RAG Pipeline

Used Generator models We evaluate a range of compact, instruction-tuned LLMs, each used in its 4-bit quantized version to reduce memory footprint and improve inference efficiency:

- **Phi-3** (3.8B parameters, 4k context length) (Abdin et al., 2024)
- **Gemma** (2B parameters, 8k context length) (Gemma Team, 2024)
- **Mistral-v0.3** (7B parameters, 32k context length) (Jiang et al., 2023)
- **Llama-2** (7B parameters, 4k context length) (Touvron et al., 2023)
- **Qwen2** (1.5B parameters, 32k context length) (Yang et al., 2024)
- **Qwen3** (8B parameters, 32k context length) (Yang et al., 2025)

These models were selected to cover a spectrum of parameter scales (from 1.5B to 8B) and context lengths while remaining feasible to run in 4-bit quantization on a single GPU, which is representative of realistic deployment constraints in academic and small-scale industrial settings. Moreover, they are all recent, instruction-tuned open-weight models, allowing us to focus the comparison on how different architectures and scales behave within the same RAG pipeline.

A detailed comparison of the model performance within the full RAG pipeline is provided in Section 3.

RAG Pipeline For answer generation we use off-the-shelf pre-trained LLMs as readers. The input to the generator is a single prompt that concatenates

- a short instruction to answer based only on the given context,
- the question
- the top- N retrieved passages ($N \leq K$) separated by simple delimiters.

We explicitly ask the model to output only the final answer phrase. The end-to-end pipeline is therefore: retrieve top- K passages with BM25, select up to N passages that fit the context window, build the prompt, generate an answer, and apply light normalization (e.g., lowercasing, trimming punctuation) before scoring.

All experiments were run in a Google Colab Pro environment equipped with a single NVIDIA L4 GPU and the high-RAM runtime option. Due to these limited computational resources, many of our evaluations are based on restricted subsets of the data and a reduced set of configurations, focusing on the most informative combinations of models, retrieval depths and prompting strategies.

2.5 Evaluation

We evaluated on the first 7900 questions of wikipedia-train.json as the validation set, and the final evaluation was on verified-wikipedia-dev.json using the official TriviaQA scorer, which reports Exact Match (EM) and token-level F_1 between predicted answers and the set of gold answers (canonical form and aliases) after standard normalization. All quantitative results in Section 3 follow this evaluation protocol.

3 Results

3.1 Retrieval Results

In our experiments, we evaluated three retrieval techniques: BM25 sparse retrieval, dense retrieval, and a hybrid method combining BM25 with a reranking model. As an evaluation metric, we used Recall@ K , where K is the number of passages passed to the generator. To compute this metric, we label a passage as relevant if it contains at least one of the gold answer aliases provided by TriviaQA; for a given question, Recall@ K is 1 if at least one relevant passage appears among the top- K retrieved passages and 0 otherwise, and we report the average of this indicator over all questions.

We compute Recall@ K on the full validation set of 7900 questions. A separate coverage analysis shows that for approximately 97.5% of the questions at least one answer alias appears in the associated evidence passages, which effectively caps the maximum achievable Recall@ K for our retrieval setup.

Across all K , the hybrid approach (BM25 + reranker) consistently outperforms both BM25 and

K	BM25	Dense	BM25 + Reranker
1	69,14%	72,61%	79,80%
3	86,51%	88,04%	90,75%
5	90,96%	92,03%	93,80%
7	93,24%	94,25%	95,30%
10	95,13%	95,61%	96,23%
20	97,44%	97,53%	97,54%

Table 1: Retrieval performance (Recall@ K) of BM25, dense retrieval and BM25 + reranker across different numbers of retrieved passages K . Values are reported as percentages.

dense retrieval, with the gains being most pronounced at small K : at $K = 1$, it reaches a Recall@1 of 79,80%, compared to 69,14% for BM25 and 72,61% for dense retrieval, indicating a stronger ability to place a relevant passage at the top rank. As K increases, all methods improve and gradually converge towards the empirical coverage ceiling; at $K = 20$, all three retrievers reach Recall@20 values around 97,5%, reflecting that most questions for which evidence exists in the corpus are successfully covered by the retrieved sets.

3.2 Generator results

In this section, we provide an initial comparison of all generator models within the RAG pipeline. All results are computed on a common subset of 400 questions taken from the TriviaQA validation set; the same 400 examples are used for every model, prompt type and retrieval depth, so as to obtain a fair comparison while keeping the computational cost manageable. To further limit GPU usage and computational cost, these preliminary experiments use a purely sparse retriever based on BM25, without the reranking component.

Model performance. Across all prompts and retrieval depths, mistral_v03 is the best-performing generator. Its highest score is 70.7 EM / 77.9 F1 in the zero-shot setting with Top-7 passages, and it also reaches 66.5 EM / 76.9 F1 in zero-shot with Top-10 and 66.0 EM / 74.4 F1 in the few-shot setting with Top-5. The second-strongest model overall is qwen3_8b, which achieves 56.0 EM / 70.0 F1 in the few-shot setting at Top-3 and stays close to this level across Top-5, Top-7 and Top-10. qwen2.1_5b forms a solid middle tier: for example, in span-extraction it reaches 58.8 EM / 66.4 F1 at Top-7. In contrast, llama_7b, phi3 and gemma obtain clearly lower scores, especially in

Zero-Shot					
Model	Top-1	Top-3	Top-5	Top-7	Top-10
gemma	21.5/29.4	36.3/44.8	33.0/41.6	30.0/39.8	13.2/24.0
llama_7b	52.7/60.2	57.7/63.1	56.0/62.0	47.6/54.7	36.75/40.33
mistral_v03	55.5/64.7	60.8/69.3	66.5/73.4	70.7/77.9	68.5/76.9
phi3	45.8/56.2	55.8/66.0	50.5/62.1	49.8/61.0	44.5/55.4
qwen2.1_5b	46.3/54.1	53.0/60.3	58.0/64.7	57.8/65.3	57.5/64.5
qwen3_8b	50.5/57.3	51.5/58.6	47.8/55.4	49.5/57.5	49.8/57.9

Few-Shot					
Model	Top-1	Top-3	Top-5	Top-7	Top-10
gemma	34.3/40.9	38.1/45.4	36.0/45.1	38.0/46.8	16.3/25.3
llama_7b	49.7/58.4	54.4/62.6	55.5/63.7	49.3/56.7	35.25/39.87
mistral_v03	57.0/65.3	60.8/69.3	66.0/74.4	65.4/74.3	64.0/74.0
phi3	43.8/52.8	50.0/61.3	43.8/57.8	44.5/57.6	39.3/51.9
qwen2.1_5b	48.0/56.3	54.0/61.7	56.0/63.7	57.0/64.9	58.5/66.3
qwen3_8b	52.8/65.8	56.0/70.0	54.8/69.2	54.0/68.7	53.0/69.2

Span-Extraction					
Model	Top-1	Top-3	Top-5	Top-7	Top-10
gemma	14.5/18.8	14.3/26.2	9.0/24.0	9.5/24.5	9.0/17.5
llama_7b	46.0/54.4	55.3/61.5	50.0/57.3	39.8/44.7	34.5/39.76
mistral_v03	47.5/56.6	59.5/65.9	62.5/69.5	64.9/72.7	66.5/73.2
phi3	39.0/47.5	54.3/63.7	51.3/62.2	49.8/59.8	46.3/54.1
qwen2.1_5b	45.3/53.4	53.3/60.4	57.0/64.4	58.8/66.4	56.5/64.7
qwen3_8b	22.5/30.1	23.3/31.3	25.3/31.7	23.3/30.7	24.0/30.5

Table 2: Comparison of model performance (EM/F1) across different prompts and the number of top- K passages passed to the model. All values are rounded to one decimal place.

the span-extraction configuration, where gemma often remains below 30 F1 and qwen3_8b also struggles compared to its zero-shot and few-shot performance.

Impact of scale. Overall, models with more parameters achieve higher EM and F1 scores and benefit more from increased retrieval depth. The largest generators in our comparison, mistral_v03 (7B, 32k context) and qwen3_8b (8B, 32k context), consistently occupy the top positions across prompts and Top-K values. The small-size qwen2.1_5b (1.5B, 32k context) remains competitive, especially in settings with more passages and a long context window (for example, span-extraction at Top-7 and Top-10). In contrast, gemma (2B, 8k context) and phi3 (3.8B, 4k context) show limited or even negative gains when moving from Top-3 to higher Top-K values; for instance, in the zero-shot setting gemma drops from 36.3 EM / 44.8 F1 at Top-3 to 13.2 EM / 24.0 F1 at Top-10. This suggests that both model capacity and context length matter: smaller or shorter-context models are more easily overwhelmed by additional passages and cannot exploit large retrieval depth as effectively.

Impact of Context Engineering. We conducted experiments to assess the impact of prompt engineering on answer extraction quality. We compared three prompting strategies: zero-shot, few-shot and span-extraction, across different model sizes and retrieval depths (Top- K). The results show a consistent pattern: zero-shot and few-shot prompting outperform span-extraction across al-

most all configurations in terms of both EM and F1. This indicates that generative prompting styles are better suited for producing accurate answers compared to span-extraction methods, especially when utilizing external retrieved context.

Across nearly all models and prompting strategies, the largest performance gains are observed when increasing Top- K , indicating that providing the model with a few highly relevant passages is particularly beneficial for answer extraction. For larger models such as Mistral-v03, performance shows a clear upward trend from Top-1 to Top-7 across all prompt types: notably, for zero-shot prompting, Mistral_v03 exhibits a clear upward trend as Top- K increases. For few-shot prompting, a similar trend is present but less pronounced. This observation motivated further investigation into whether the positive trend for zero-shot persists with increasing retrieval depth in subsequent experiments: further exploration of optimal Top- K values could yield additional performance gains. This trend demonstrates that larger-scale models benefit substantially from increased retrieval depth.

3.3. Detailed analysis of mistral_v03

Building on these findings, we selected the best-performing model, Mistral_v03, and we evaluated it in a zero-shot setting on 2,000 questions from the validation set, using BM25 + reranker retriever to further improve performance; the size of this subset is limited by computational time and hardware constraints, but it provides more robust estimates than the initial 400-question comparison. We systematically varied the number of retrieved passages inserted into the prompt (Top-5, 7, 10, 15, 20, 25, 30) in order to test whether the upward trend in performance observed at lower values of K persists, to identify the point at which additional passages no longer bring significant gains, and to better understand the trade-off between supplying richer contextual evidence and introducing noise through excessively long prompts.

The overall goal of this analysis is to identify the most effective combination of retriever depth and prompting configuration, that is, the best “recipe” for achieving the highest possible performance with mistral_v03 in our RAG setup.

Table 3 and Figure 1 summarize the results. Both EM and F1 improve as we increase Top- K from 5 to 10: EM rises from 67,95% to 69,59%, while F1 increases from 75,91% to a peak of 77,34%. Beyond Top-10, both metrics plateau: EM fluctuates

Top- K	EM / F1
5	67,95% / 75,91%
7	68,30% / 75,76%
10	69,59% / 77,34%
15	69,45% / 77,20%
20	69,85% / 77,15%
25	69,54% / 77,11%
30	69,57% / 76,83%

Table 3: mistral_v03 performance (EM/F1) on 2 000 validation questions with BM25 + reranker for different numbers of retrieved passages Top- K .

in a narrow range between 69,45% and 69,85%, and F1 remains between 76,83% and 77,20%. This suggests that mistral_v03 can effectively exploit a moderate increase in retrieved context, but additional passages beyond circa 10–15 bring only marginal gains and sometimes a slight decrease, likely because the extra information adds redundancy and noise without introducing substantially new evidence. Overall, these results indicate that retrieval depths around Top-10 provide a good trade-off between coverage and prompt length for mistral_v03 in our RAG setup.

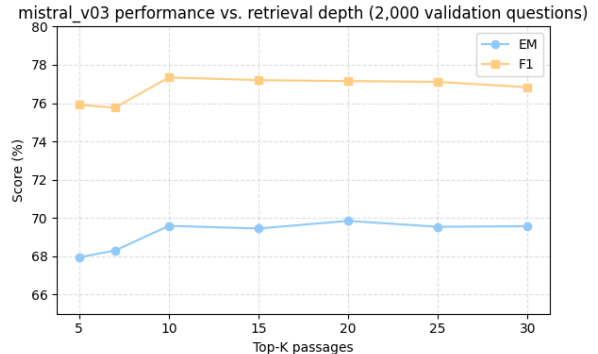


Figure 1: mistral_v03 performance (EM and F1) on 2 000 validation questions as a function of retrieval depth Top- K . The y-axis is zoomed to the 65–80% range to highlight the trend.

For the final evaluation on the full 7 900-question validation set (and later on test), we fix the configuration to BM25 + reranker with Top-10 passages and zero-shot prompting for mistral_v03, as this setting offers the best overall F1 and near-optimal EM while keeping inference cost manageable.

3.4 Final evaluation on validation and test sets.

We conclude by evaluating our best RAG configuration on the full validation and test sets. The final setup combines BM25 retrieval over Wikipedia

passages chunked into 350-word segments with 70-word overlap, a cross-encoder reranker applied to the top BM25 candidates (BM25 + reranker), and mistral_v03 as generator in a zero-shot setting; for each question, the generator receives the Top-10 reranked passages concatenated into the prompt.

Dataset	EM	F1
Validation (7 900)	76,10%	83,89%
Test (318)	83,16%	90,20%

Table 4: Final performance of the selected RAG configuration (BM25 + reranker, Top-10 passages, zero-shot mistral_v03) on validation and test sets. Values are reported as percentages.

On the full validation split of 7 900 questions, this configuration reaches approximately 76,10% exact match and 83,89% F1, confirming that the trends observed on the 2 000-question subset extend to the entire validation set. When applied to the held-out test set of 318 human-validated questions, the same model achieves 83,16% EM and 90,20% F1, indicating strong generalisation and suggesting that, under this RAG setup, mistral_v03 can reliably recover and synthesise the correct answers when provided with a small set of high-quality retrieved passages.

For comparison, using the same zero-shot prompt with mistral_v03 but without any retrieved passages (Top- $K = 0$) on the test set yields 73,06% EM and 79,32% F1. This gap of roughly ten points in both EM and F1 highlights the substantial contribution of the retrieval component to the overall effectiveness of the RAG pipeline.

4 Conclusion

In this work, we presented a complete retrieval-augmented generation pipeline for open-domain question answering on TriviaQA, focusing on the Wikipedia setting. Starting from raw evidence documents, we designed a preprocessing and chunking strategy based on 350-word passages with 70-word overlap, and compared several retrieval methods as well as multiple compact instruction-tuned LLMs. Our analysis of retrieval performance showed that a hybrid approach combining BM25 with a cross-encoder reranker consistently achieves the highest Recall@K, and therefore constitutes a strong backbone for RAG in this scenario.

On top of this retrieval layer, we carried out a systematic comparison of six generator models

and three prompting strategies. The experiments on a shared pool of 400 validation questions highlighted mistral_v03 as the best-performing model across prompts and Top-K values, and indicated that zero-shot prompting is generally more effective than span-extraction and competitive with few-shot prompting. A subsequent, more detailed study on 2 000 validation questions confirmed that mistral_v03 benefits from increasing retrieval depth up to around ten passages, after which performance saturates and very deep retrieval yields only marginal gains.

Guided by these findings, we selected a final RAG configuration that uses BM25 + reranker over Wikipedia chunks, mistral_v03 as zero-shot generator, and the Top-10 reranked passages concatenated into the prompt. This setup achieves 76,10% EM and 83,89% F1 on the full 7 900-question validation set, and 83,16% EM and 90,20% F1 on the held-out test set of 318 human-validated questions. These results suggest that, under realistic computational constraints, a carefully tuned RAG pipeline with a compact open-weight model can deliver strong performance on TriviaQA without any task-specific fine-tuning. Future work could explore more advanced dense retrievers, joint training of retriever and generator, and extensions beyond the Wikipedia-only setting to further improve robustness and coverage.

A Appendix: Prompt templates

This appendix reports the exact prompt templates used in our experiments. In all cases, [CONTEXT_BLOCK] denotes the concatenated retrieved passages and [QUESTION] the input question.

In all cases, [CONTEXT_BLOCK] is obtained by concatenating the top- K retrieved passages (350-word Wikipedia chunks with 70-word overlap) in ranking order, separated by blank lines.

A.1 Prompt v1: Zero-shot

You answer ONE trivia question at a time.
Follow these rules STRICTLY:

1. Use ONLY the information in the passages.
2. Output ONLY the final answer.
3. The answer must be ONE or TWO WORDS.
4. Do NOT write full sentences.
5. Do NOT repeat or paraphrase the question.
6. Do NOT add explanations or extra words.
7. Do NOT write any new questions.
8. If the answer is not in the passages,

output exactly: I don't know
Passages: [CONTEXT_BLOCK]
Question: [QUESTION]
Final answer (ONE OR TWO WORDS ONLY):

A.2 Prompt v2: Few-shot

You answer trivia questions based on passages.
Always respond with ONE or TWO WORDS only.
Examples:
Passages:
Paris is the capital and most populous city of France.
Question: What is the capital of France?
Answer: Paris
Passages:
Albert Einstein developed the theory of relativity.
Question: Who developed the theory of relativity?
Answer: Einstein
Passages:
Ferdinand Magellan led the first expedition to circumnavigate the globe.
Question: Who led the first expedition to sail around the world?
Answer: Magellan
Now answer the next question in the SAME FORMAT.
Passages: [CONTEXT_BLOCK]
Question: [QUESTION]
Answer (ONE OR TWO WORDS ONLY):

A.3 Prompt v3: Span-extraction

You answer trivia questions using the passages below.
Your answer must be a short span (ONE or TWO WORDS) copied from the passages.
Choose the shortest, most natural alias (e.g., Tolkien, Magellan, New York).
Do NOT invent new words.
Do NOT explain your answer.
If the answer is not present in the passages, output exactly: I don't know
Passages: [CONTEXT_BLOCK]
Question: [QUESTION]
Final answer (ONE OR TWO WORDS, copied from the passages):

References

- Marah Abdin, Sam Ade Jacobs, Ammar Ahmad Awan, Jyoti Aneja, Ahmed Awadallah, Hany Awadalla, et al. 2024. *Phi-3 technical report: A highly capable language model locally on your phone*.
- Danqi Chen, Adam Fisch, Jason Weston, and Antoine Bordes. 2017. Reading wikipedia to answer open-domain questions. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 1870–1879.
- Gemma Team. 2024. *Gemma: Open models based on gemini research and technology*.
- Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de Las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Léo Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2023. Mistral 7b. *arXiv preprint arXiv:2310.06825*.
- Mandar Joshi, Eunsol Choi, Daniel S. Weld, and Luke Zettlemoyer. 2017. Triviaqa: A large scale distantly supervised challenge dataset for reading comprehension. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 1601–1611.
- Tri Nguyen, Mir Rosenberg, Xia Song, Jianfeng Gao, Saurabh Tiwary, Rangan Majumder, and Li Deng. 2016. MS MARCO: A human generated machine reading comprehension dataset. In *Proceedings of the Workshop on Cognitive Computing and NLP*.
- Nils Reimers and Iryna Gurevych. 2019. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of EMNLP-IJCNLP*, pages 3982–3992.
- Stephen Robertson and Hugo Zaragoza. 2009. The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389.
- Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.
- An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, et al. 2024. Qwen2 technical report. *arXiv preprint arXiv:2407.10671*.
- Link of the project's notebook hosted on Google Colab:
- Project:
https://colab.research.google.com/drive/1_stgRKaDRQYkhEhGg6pLPGe48qnkTHef?usp=sharing
- Data Preprocessing:
<https://colab.research.google.com/drive/12sLHN21H-Z8kdu29Zms1zZCN2CnZ180t?usp=sharing>